STUDENT ACADEMIC PROGRESSION & EARLY DROPOUT RISK DETECTION SYSTEM

Database Management System — Semester Project Report

Field	Details
Submitted To	Mr. Ihtisham Ullah
Department	Faculty of Computing
University	Riphah International University Islamabad I-14
Session	Fall 2026
Section	BSCS 4-2
Student Name	Muhammad Bin Riaz
SAP ID	65256
Tool Used	MySQL Workbench
Language	SQL (MySQL)

Table of Contents

1. Introduction

The Student Academic Progression & Early Dropout Risk Detection System is a database-driven application designed to monitor student performance and identify at-risk students proactively. Traditional university systems merely store academic data without providing any analytical insight, leaving advisors to discover problems too late.

This system addresses that gap by tracking GPA trends, attendance patterns, and course failure rates using structured SQL queries executed against a normalized

relational database. Advisors and department heads can use the system's output to take timely, targeted action before a student drops out.

2. Objectives

- Design a structured, normalized relational database (up to 3NF)
- Ensure data integrity through primary keys, foreign keys, and check constraints
- Track student enrollment, attendance, and semester-wise GPA
- Detect GPA decline and dangerously low attendance using SQL
- Identify course failures (two or more F grades) as a dropout indicator
- Generate analytical reports for academic advisors and department heads
- Demonstrate advanced SQL: JOINs, subqueries, aggregations, and views

3. Scope of the Project

Target Users

- Academic Admin
- Department Heads
- Academic Advisors

Key Features

- Manage student and department records
- Track course enrollments with grades
- Record daily attendance
- Store semester-wise GPA history
- Identify students with GPA decline over consecutive semesters
- Generate risk reports (High / Moderate / Low)

System Boundaries

The system manages academic data only. Finance, HR, and library systems are out of scope.

4. Entity Relationship Overview

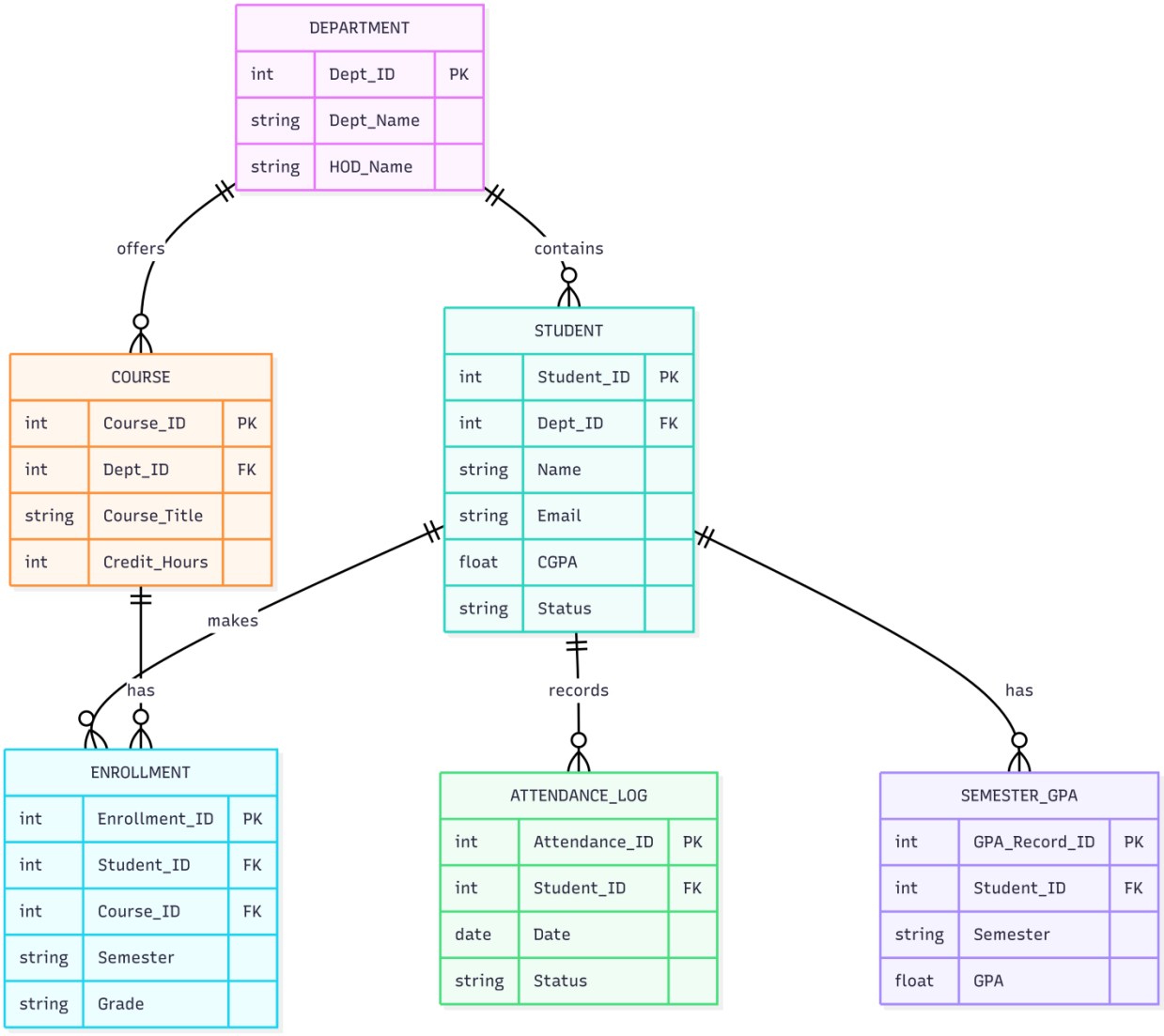
The database consists of six main entities:

Entity	Primary Key	Description
Department	Dept_ID	Stores department name and HOD
Student	Student_ID	Stores student profile linked to a department
Course	Course_ID	Stores course title, credit hours, linked to dept
Enrollment	Enrollment_ID	Links a student to a course in a semester with grade
Attendance_Log	Attendance_ID	Records daily attendance status per student
Semester_GPA	GPA_Record_ID	Stores per-semester GPA for each student

Relationships

- One Department has many Students
- One Department offers many Courses
- One Student has many Enrollments
- One Course has many Enrollments
- One Student has many Attendance_Log records
- One Student has many Semester_GPA records

5. ERD Diagram



6. Database Schema

The database consists of 6 related tables. All tables are implemented in MySQL.

6.1 DEPARTMENT

```
CREATE TABLE Department (
    Dept_ID     INT PRIMARY KEY AUTO_INCREMENT,
    Dept_Name   VARCHAR(100) NOT NULL,
    HOD_Name    VARCHAR(100) NOT NULL
```

```
);
```

6.2 STUDENT

```
CREATE TABLE Student (
   Student_ID  INT PRIMARY KEY AUTO_INCREMENT,
   Dept_ID     INT NOT NULL,
   Name        VARCHAR(100) NOT NULL,
   Email       VARCHAR(100) UNIQUE NOT NULL,
   CGPA        DECIMAL(3,2) DEFAULT 0 CHECK (CGPA BETWEEN 0 AND 4),
   Status      VARCHAR(20) DEFAULT 'Active' CHECK (Status IN
('Active','Probation','Dropout')),
   FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

6.3 COURSE

```
CREATE TABLE Course (
   Course_ID    INT PRIMARY KEY AUTO_INCREMENT,
   Dept_ID      INT NOT NULL,
   Course_Title VARCHAR(100) NOT NULL,
   Credit_Hours INT CHECK (Credit_Hours > 0),
   FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

6.4 ENROLLMENT

```
CREATE TABLE Enrollment (
   Enrollment_ID INT PRIMARY KEY AUTO_INCREMENT,
   Student_ID    INT NOT NULL,
   Course_ID     INT NOT NULL,
   Semester      VARCHAR(20) NOT NULL,
   Grade         VARCHAR(2),
   FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID) ON DELETE
CASCADE,
   FOREIGN KEY (Course_ID)  REFERENCES Course(Course_ID)  ON DELETE
CASCADE,
   UNIQUE (Student_ID, Course_ID, Semester)
);
```

6.5 ATTENDANCE_LOG

```
CREATE TABLE Attendance_Log (
   Attendance_ID INT PRIMARY KEY AUTO_INCREMENT,
   Student_ID    INT NOT NULL,
   Att_Date      DATE NOT NULL,
   Status        VARCHAR(10) CHECK (Status IN ('Present','Absent','Late')),
   FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID) ON DELETE
CASCADE
);
```

6.6 SEMESTER_GPA

```
CREATE TABLE Semester_GPA (
   GPA_Record_ID INT PRIMARY KEY AUTO_INCREMENT,
   Student_ID    INT NOT NULL,
   Semester      VARCHAR(20) NOT NULL,
   GPA           DECIMAL(3,2) CHECK (GPA BETWEEN 0 AND 4),
   FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID) ON DELETE
CASCADE
);
```

7. Normalization

Normal Form	Rule Applied	Example in This System
1NF	All attributes contain atomic (single) values; no repeating groups.	Each row in Enrollment holds exactly one course-student-semester combination. No comma-separated course lists.
2NF	No partial dependencies — every non-key attribute depends on the whole primary key.	In Semester_GPA, GPA depends on (Student_ID + Semester) — the full composite key, not just Student_ID alone.
3NF	No transitive dependencies — non-key attributes depend only on the primary key.	Student stores Dept_ID (FK), not Dept_Name. Dept_Name lives only in Department, eliminating transitive dependency.

Benefits of Normalization

- Reduces data redundancy — no duplicate department names stored in Student
- Improves data consistency — updating a department name requires changing one row
- Simplifies updates and deletions — cascade rules handle child records automatically

8. Constraints

Constraint	Table(s)	Purpose
PRIMARY KEY	All tables	Unique identification of every record
FOREIGN KEY	Student, Course, Enrollment, Attendance_Log, Semester_GPA	Referential integrity — prevents orphan records
NOT NULL	Name, Email, Dept_ID, Course_Title, Semester, Status (Att)	Prevents missing required data
UNIQUE	Student.Email	No two students can share the same email address
UNIQUE composite	Enrollment (Student_ID, Course_ID, Semester)	A student cannot enroll in the same course twice in one semester
CHECK (CGPA)	Student	CGPA must be between 0.0 and 4.0
CHECK (GPA)	Semester_GPA	GPA must be between 0.0 and 4.0
CHECK (Status)	Student	Status must be Active, Probation, or Dropout
CHECK (Attendance)	Attendance_Log	Status must be Present, Absent, or Late

Constraint	Table(s)	Purpose
CHECK (Credits)	Course	Credit hours must be greater than 0
ON DELETE CASCADE	Enrollment, Attendance_Log, Semester_GPA	Deleting a student automatically removes their related records
DEFAULT	Student.CGPA, Student.Status	Auto-sets sensible default values on insert

9. Risk Detection Logic

The system uses SQL-based analysis to classify each student into one of three risk levels:

Risk Indicator	Threshold / Condition
GPA Decline	2 or more consecutive semesters of decreasing GPA
Low Attendance	Attendance percentage falls below 60%
Course Failure	2 or more F grades recorded in Enrollment
Low CGPA	CGPA below 2.0 after completing the 2nd semester

Risk Level	Definition
High Risk	Student meets 2 or more of the above indicators
Moderate Risk	Student meets exactly 1 indicator
Low Risk	Student meets none of the indicators

10. SQL Queries — DDL (CREATE)

The DDL script creates all six tables. See Section 6 for the full CREATE TABLE statements. A combined DDL execution order (respecting foreign key dependencies) is shown below:

```
-- Execute in this order to respect FK dependencies
-- 1. Department (no FK)
-- 2. Student    (FK -> Department)
-- 3. Course     (FK -> Department)
-- 4. Enrollment (FK -> Student, Course)
-- 5. Attendance_Log (FK -> Student)
-- 6. Semester_GPA   (FK -> Student)
```

11. SQL Queries — DML (INSERT)

Sample data populates each table for testing and demonstration.

```
-- Departments
INSERT INTO Department VALUES (1, 'Computer Science', 'Dr. Tariq');
INSERT INTO Department VALUES (2, 'Software Engineering', 'Dr. Amna');

-- Students
INSERT INTO Student VALUES (101, 1, 'Muhammad Bin Riaz',
'muhammad@riphah.edu.pk', 3.5, 'Active');
INSERT INTO Student VALUES (102, 1, 'Ahmed Ali',     'ahmed@riphah.edu.pk',
1.8, 'Probation');
INSERT INTO Student VALUES (103, 2, 'Ayesha Noor',   'ayesha@riphah.edu.pk',
2.9, 'Active');
INSERT INTO Student VALUES (104, 2, 'Bilal Khan',    'bilal@riphah.edu.pk',   1.5,
'Probation');

-- Courses
INSERT INTO Course VALUES (201, 1, 'Database Systems',  3);
INSERT INTO Course VALUES (202, 1, 'Data Structures',   4);
INSERT INTO Course VALUES (203, 2, 'Software Design',   3);

-- Enrollments
INSERT INTO Enrollment VALUES (301, 101, 201, 'Fall-2025', 'A');
INSERT INTO Enrollment VALUES (302, 102, 201, 'Fall-2025', 'F');
INSERT INTO Enrollment VALUES (303, 103, 202, 'Fall-2025', 'B');
```

```
INSERT INTO Enrollment VALUES (304, 104, 201, 'Fall-2025', 'F');
INSERT INTO Enrollment VALUES (305, 104, 202, 'Fall-2025', 'F');

-- Attendance_Log (sample records)
INSERT INTO Attendance_Log VALUES (401, 101, '2025-10-01', 'Present');
INSERT INTO Attendance_Log VALUES (402, 102, '2025-10-01', 'Absent');
INSERT INTO Attendance_Log VALUES (403, 103, '2025-10-01', 'Present');
INSERT INTO Attendance_Log VALUES (404, 104, '2025-10-01', 'Absent');

-- Semester_GPA
INSERT INTO Semester_GPA VALUES (501, 101, 'Spring-2025', 3.7);
INSERT INTO Semester_GPA VALUES (502, 101, 'Fall-2025',   3.5);
INSERT INTO Semester_GPA VALUES (503, 102, 'Spring-2025', 2.8);
INSERT INTO Semester_GPA VALUES (504, 102, 'Fall-2025',   1.8);
INSERT INTO Semester_GPA VALUES (505, 104, 'Spring-2025', 2.0);
INSERT INTO Semester_GPA VALUES (506, 104, 'Fall-2025',   1.5);
```

12. SQL Queries — Joins, Subqueries & Aggregation

12.1 INNER JOIN — Full Student Report

Retrieve each student's name, course, grade, and attendance status.

```
SELECT s.Name, c.Course_Title, e.Grade, e.Semester
FROM Student s
INNER JOIN Enrollment e ON s.Student_ID = e.Student_ID
INNER JOIN Course c     ON e.Course_ID  = c.Course_ID
ORDER BY s.Name, e.Semester;
```

12.2 LEFT JOIN — Students With No Enrollments

Identify students who are registered but not enrolled in any course this semester.

```
SELECT s.Student_ID, s.Name, s.Email
FROM Student s
LEFT JOIN Enrollment e ON s.Student_ID = e.Student_ID
WHERE e.Enrollment_ID IS NULL;
```

12.3 Subquery — Students With GPA Below Average

Find students whose CGPA falls below the department average.

```
SELECT Name, CGPA
FROM Student
WHERE CGPA < (SELECT AVG(CGPA) FROM Student)
ORDER BY CGPA ASC;
```

12.4 Aggregation — Department Performance Summary

Count students and compute average CGPA per department.

```
SELECT d.Dept_Name,
    COUNT(s.Student_ID) AS Total_Students,
    ROUND(AVG(s.CGPA), 2) AS Avg_CGPA,
    MIN(s.CGPA) AS Min_CGPA,
    MAX(s.CGPA) AS Max_CGPA
FROM Department d
LEFT JOIN Student s ON d.Dept_ID = s.Dept_ID
GROUP BY d.Dept_Name
ORDER BY Avg_CGPA DESC;
```

12.5 Risk Query — High Risk Students (Attendance < 60%)

Identify students whose attendance rate is critically low. Absence count calculated from Attendance_Log.

```
SELECT s.Student_ID, s.Name,
    ROUND(
      (COUNT(CASE WHEN a.Status = 'Present' THEN 1 END) * 100.0) /
COUNT(a.Attendance_ID),
      1
    ) AS Attendance_Pct
FROM Student s
JOIN Attendance_Log a ON s.Student_ID = a.Student_ID
GROUP BY s.Student_ID, s.Name
HAVING (COUNT(CASE WHEN a.Status = 'Present' THEN 1 END) * 100.0) /
COUNT(a.Attendance_ID) < 60
ORDER BY Attendance_Pct ASC;
```

12.6 Risk Query — Students With 2+ F Grades

Flag students who have failed two or more courses — a strong dropout indicator.

```
SELECT s.Student_ID, s.Name, COUNT(e.Grade) AS F_Count
FROM Student s
JOIN Enrollment e ON s.Student_ID = e.Student_ID
WHERE e.Grade = 'F'
GROUP BY s.Student_ID, s.Name
HAVING COUNT(e.Grade) >= 2
ORDER BY F_Count DESC;
```

12.7 Consecutive GPA Decline Detection

Detect students whose GPA decreased in two consecutive semesters using a self-join on Semester_GPA.

```
SELECT g1.Student_ID, s.Name, g1.Semester AS Sem1, g1.GPA AS GPA1,
      g2.Semester AS Sem2, g2.GPA AS GPA2
FROM Semester_GPA g1
JOIN Semester_GPA g2 ON g1.Student_ID = g2.Student_ID
JOIN Student s       ON g1.Student_ID = s.Student_ID
WHERE g2.GPA < g1.GPA
  AND g1.Semester < g2.Semester
ORDER BY g1.Student_ID;
```

12.8 Comprehensive Risk Report — All Indicators

Combines GPA, attendance, and failure data into a single risk-level report using CASE.

```
SELECT s.Student_ID, s.Name, s.CGPA,
   (SELECT COUNT(*) FROM Enrollment e WHERE e.Student_ID = s.Student_ID
AND e.Grade = 'F') AS F_Grades,
   ROUND(
     (SELECT COUNT(*) FROM Attendance_Log a WHERE a.Student_ID =
s.Student_ID AND a.Status = 'Present') * 100.0
     / NULLIF((SELECT COUNT(*) FROM Attendance_Log a2 WHERE
a2.Student_ID = s.Student_ID), 0)
```

```
    , 1) AS Attendance_Pct,
   CASE
     WHEN s.CGPA < 2.0
       AND (SELECT COUNT(*) FROM Enrollment e WHERE e.Student_ID =
s.Student_ID AND e.Grade = 'F') >= 2
       THEN 'HIGH RISK'
     WHEN s.CGPA < 2.5
       OR  (SELECT COUNT(*) FROM Enrollment e WHERE e.Student_ID =
s.Student_ID AND e.Grade = 'F') >= 1
       THEN 'MODERATE RISK'
     ELSE 'LOW RISK'
   END AS Risk_Level
FROM Student s
ORDER BY Risk_Level, s.CGPA ASC;
```

13. SQL Views

Views encapsulate complex queries for easy reuse by advisors and reporting tools.

13.1 View — At-Risk Students

```
CREATE OR REPLACE VIEW vw_AtRisk_Students AS
SELECT s.Student_ID, s.Name, s.CGPA,
   COUNT(CASE WHEN e.Grade = 'F' THEN 1 END) AS Failed_Courses,
   ROUND(
     COUNT(CASE WHEN a.Status = 'Present' THEN 1 END) * 100.0 /
NULLIF(COUNT(a.Attendance_ID), 0)
   , 1) AS Attendance_Pct
FROM Student s
LEFT JOIN Enrollment e    ON s.Student_ID = e.Student_ID
LEFT JOIN Attendance_Log a ON s.Student_ID = a.Student_ID
GROUP BY s.Student_ID, s.Name, s.CGPA
HAVING s.CGPA < 2.5
   OR COUNT(CASE WHEN e.Grade = 'F' THEN 1 END) >= 2
   OR (COUNT(CASE WHEN a.Status = 'Present' THEN 1 END) * 100.0 /
NULLIF(COUNT(a.Attendance_ID), 0)) < 60;
```

13.2 View — Department Summary

```
CREATE OR REPLACE VIEW vw_Dept_Summary AS
SELECT d.Dept_Name,
   COUNT(s.Student_ID)  AS Total_Students,
   ROUND(AVG(s.CGPA),2) AS Avg_CGPA,
   COUNT(CASE WHEN s.Status = 'Probation' THEN 1 END) AS On_Probation,
   COUNT(CASE WHEN s.Status = 'Dropout'   THEN 1 END) AS Dropped_Out
FROM Department d
LEFT JOIN Student s ON d.Dept_ID = s.Dept_ID
GROUP BY d.Dept_Name;
```

13.3 View — Semester GPA Trend

```
CREATE OR REPLACE VIEW vw_GPA_Trend AS
SELECT s.Name, g.Semester, g.GPA,
   LAG(g.GPA) OVER (PARTITION BY g.Student_ID ORDER BY g.Semester) AS
Prev_GPA,
   g.GPA - LAG(g.GPA) OVER (PARTITION BY g.Student_ID ORDER BY
g.Semester) AS GPA_Change
FROM Semester_GPA g
JOIN Student s ON g.Student_ID = s.Student_ID
ORDER BY s.Name, g.Semester;
```

14. Expected Outputs

Output Report	Generated By	Used By
Student academic summary report	INNER JOIN (Student + Enrollment + Course)	Academic Admin
Semester GPA trend analysis	vw_GPA_Trend view (requires MySQL 8.0+)	Advisors
Attendance percentage report	Aggregation on Attendance_Log	Department Heads
High-risk student list	Comprehensive Risk Report Query (12.8)	Advisors
Department-wise performance summary	vw_Dept_Summary view	HODs, Admin

Output Report	Generated By	Used By
Students with consecutive GPA decline	Self-join on Semester_GPA (12.7)	Advisors
Students with low attendance AND poor grades	Combined risk query (12.5 + 12.6)	Advisors
Students with no enrollments (LEFT JOIN)	LEFT JOIN query (12.2)	Academic Admin

15. Tools & Technology

Component	Technology Used
Database Engine	MySQL 8.0
Query Language	SQL (MySQL dialect)
Development Tool	MySQL Workbench
ERD Design	Mermaid (mermaid.live)
Version Control	Git + GitHub
Documentation	Microsoft Word

16. GitHub Repository

All SQL scripts, views, and the project report are hosted on GitHub for version control and submission

Item	Details
Repository URL	https://github.com/muhammadbinriaz/DB-Project
Contents	Scripts of sql, ERD diagram, Project report PDF
Branch	main
README	Project explanations

17. Conclusion

The Student Academic Progression & Early Dropout Risk Detection System successfully demonstrates how a properly designed relational database can go beyond simple data storage to provide actionable analytical insights.

Through the design of six normalized tables (up to 3NF), rigorous constraint enforcement, and a suite of advanced SQL queries — including multi-table JOINs, correlated subqueries, aggregation functions, and reusable views — the system enables academic advisors to proactively identify students at risk of dropping out.

The risk detection framework classifies students as High, Moderate, or Low risk based on measurable indicators (GPA decline, low attendance, course failures, and CGPA thresholds), providing a data-driven foundation for timely academic interventions.

This project demonstrates mastery of database design, normalization theory, constraint management, and advanced MySQL — fulfilling all requirements of the DBMS semester project.